

LLM Fine Tuning & Optimization

UNIT -IV

Contents -

- Optimization Techniques
- Optimizers: Adam, AdamW, Adafactor
- Gradient clipping, learning rate scheduling, and mixed precision training,
- Knowledge Distillation and Quantization, Pruning and Compression for deployment efficiency

Optimization Techniques – Adam Optimizer

- In machine learning and deep learning, a model learns by adjusting its weights to reduce error (loss).
An optimizer is the algorithm that decides how the weights should change to make the model better.
- One of the most popular optimizers is Adam.

What is Adam Optimizer?

- Adam (Adaptive Moment Estimation) is an optimizer used to update the weights of a neural network during training.

In simple words:

- Adam helps the model learn faster and more accurately by automatically adjusting the learning rate.

Why Do We Use Adam?

- When training a neural network, we want to:
- Reduce the error (loss)
- Update the weights correctly
- Reach the best solution quickly
- Adam helps achieve this by using information from previous gradients.

Simple Idea Behind Adam

- Adam keeps track of two things:
- Past gradients (momentum)
 - Helps the model move in the correct direction.
- Squared gradients (adaptive learning rate)
 - Adjusts the learning speed for each parameter.
- Because of this, Adam learns efficiently and avoids large mistakes.

Simple Example

- Imagine a student trying to reach the lowest point of a hill (minimum loss).
- Gradient $\rightarrow$ shows which direction to go
- Learning rate $\rightarrow$ decides how big the step is
- Adam optimizer $\rightarrow$ helps the student choose smart steps using past experience
- So the student reaches the lowest point faster.

Advantages of Adam

- ✓ Easy to use
- ✓ Fast learning
- ✓ Works well with large datasets
- ✓ Automatically adjusts learning rate
- ✓ Very popular in deep learning

Where Adam is Used

- Adam is commonly used in training large AI models such as:
- BERT
- GPT
- T5
- These models are used for tasks like translation, chatbots, text generation, and question answering.

Optimization Techniques – AdamW Optimizer

- AdamW is an improved version of the Adam Optimizer used to train deep learning models.
It is widely used in modern AI models, especially in transformer-based models.

What is AdamW?

- AdamW (Adam with Weight Decay) is an optimizer that updates model weights while also preventing the model from becoming too complex.
- In simple words:
- AdamW helps the model learn efficiently and avoids overfitting by controlling the size of weights.

Why AdamW is Needed?

- In the original Adam optimizer, weight decay (regularization) was not handled correctly.
- Problems with Adam:
- Weights can become very large.
- The model may overfit the training data.
- AdamW solves this problem by applying weight decay separately from the gradient update.

Simple Idea Behind AdamW

- AdamW works in two steps:
- Update weights using Adam optimization
- Apply weight decay to reduce large weights
- This helps the model:
- Learn properly
- Avoid overfitting
- Generalize better to new data

Simple Example

- Imagine a student preparing for exams:
- Adam optimizer $\rightarrow$ helps the student learn quickly.
- Weight decay in AdamW $\rightarrow$ reminds the student not to memorize everything blindly but focus on important concepts.
- So the student performs better in real exams, not just practice tests.

Advantages of AdamW

- ✓ Better regularization
- ✓ Reduces overfitting
- ✓ Works well with transformer models
- ✓ More stable training
- ✓ Better performance in modern deep learning

Where AdamW is Used

- AdamW is widely used in training Large Language Models (LLMs) such as:
- BERT
- GPT
- T5
- These models are used in chatbots, translation, text generation, and summarization.

Optimization Techniques – **Adafactor Optimizer**

- Adafactor is an optimization algorithm used to train very large neural networks efficiently.
- When training deep learning models, the optimizer updates the model weights to reduce error.
Adafactor works similarly to the Adam Optimizer, but it uses much less memory.
- Large AI models contain millions or billions of parameters, and storing gradient information for each parameter requires a lot of memory.
Adafactor solves this problem by reducing the memory required during training.

Example

- When training large models such as:
- BERT
- T5
- Adafactor helps train the model **efficiently without using too much GPU memory.**

Optimization Techniques - Gradient Clipping

- Gradient clipping is a technique used to control very large gradients during training.
- During training, gradients help update the model weights. Sometimes gradients become very large, which causes unstable learning. This problem is called exploding gradients.
- Gradient clipping limits the size of gradients so that the model updates weights in a controlled way.

Example

- Imagine a student walking down a hill to reach the lowest point.
- If the step is too large, the student may fall.
- Gradient clipping controls the step size, so the student moves safely.

Optimization Techniques - Learning Rate Scheduling

- Learning rate scheduling is a method where the learning rate changes during training.
- The learning rate controls how fast a model learns.
- At the beginning, we use a larger learning rate so the model learns quickly.
- Later, we use a smaller learning rate for fine adjustments.

Example

- When learning to ride a bicycle:
- At first, you make large movements.
- Later, you make small corrections.
- Learning rate scheduling works in the same way.

Optimization Techniques - Mixed Precision Training

- Mixed precision training uses different numerical precisions during training.
- Normally models use 32-bit numbers (FP32).
Mixed precision training uses both:
- 16-bit precision (FP16)
- 32-bit precision (FP32)
- This helps reduce memory usage and increase training speed.

Example

- Large models like:
- GPT
- BERT
- use mixed precision training to train faster on GPUs.

Optimization Techniques - Knowledge Distillation

- Knowledge distillation is a technique where a large model teaches a smaller model.
- Large models may be too heavy for mobile devices.
Using knowledge distillation, we create smaller models with similar performance.

Optimization Techniques - Quantization

- Normally model weights are stored using 32-bit numbers. Quantization converts them into lower precision numbers, such as:
 - 16-bit
 - 8-bit
- This reduces memory usage and computation cost.

Example

- Quantization is useful when deploying AI models on:
- mobile phones
- IoT devices
- embedded systems

Pruning and Compression for Deployment Efficiency

- When deep learning models are trained, they often become very large and complex. Such models require high memory, large storage, and powerful hardware. This makes it difficult to deploy them on mobile devices, edge devices, or embedded systems.
- To solve this problem, techniques like pruning and model compression are used. These methods reduce model size and computation cost while maintaining good accuracy.

Methods **reduce model size and computation cost**

1. Model Pruning
2. Model Compression

1. Model Pruning

Model Pruning

Definition

- Model pruning is the process of removing unnecessary weights, neurons, or connections from a trained neural network to make the model smaller and more efficient.
- Deep neural networks often have many redundant parameters. Removing these parameters helps reduce the model complexity and computational cost.

Why Pruning is Needed

- Large neural networks may contain millions or billions of parameters. Many of these parameters contribute very little to the final output.
- Problems with large models:
- High memory consumption
- Slow inference time
- High power consumption
- Difficult deployment on mobile devices

How Pruning Works

During pruning:

1. The model is trained normally.
2. The importance of each weight is evaluated.
3. Weights with very small values or low importance are removed.
4. The model is retrained (fine-tuned) to recover any lost accuracy.

This process makes the network sparser but still effective.

Types of Pruning

1. Weight Pruning (Unstructured Pruning)

- This technique removes **individual weights** that have very small values.

Example:

If a weight value is close to **0**, it contributes very little to prediction, so it can be removed.

Advantages:

- High compression ratio
- Reduces number of parameters

2. Structured Pruning

- Structured pruning removes **entire structures** from the network such as:
 - neurons
 - filters
 - channels
 - layers

Advantages:

- Easier for hardware acceleration
- Improves inference speed

3. Neuron Pruning

- This removes **entire neurons** from the neural network.
- If a neuron contributes very little to the output, it can be removed along with its connections.

Example of Pruning

- Consider a neural network with **1000 connections**.
- After pruning:
- Only **600 important connections remain**
- **400 unnecessary connections are removed**

Result:

- Smaller model
- Faster prediction
- Less memory usage

2. Model Compression

Definition

- Model compression is a group of techniques used to reduce the size of a deep learning model while maintaining its performance.
- Large AI models like:
- BERT
- GPT
- are powerful but very large, making deployment difficult. Compression techniques help make these models lighter and faster.

Why Model Compression is Important

- Deep learning models are often deployed in environments with **limited resources**, such as:
- Mobile phones
- IoT devices
- Edge computing systems
- Embedded systems
- These devices have **limited memory, processing power, and battery life**.
- Model compression allows AI models to **run efficiently on such devices**.

Common Model Compression Techniques

1. Quantization
2. Knowledge Distillation
3. Weight Sharing
4. Pruning

1. Quantization

- Quantization reduces the **precision of numbers used to represent model parameters**.
- Normally weights are stored in **32-bit floating point (FP32)** format.
- Quantization converts them into:
 - 16-bit (FP16)
 - 8-bit integers (INT8)
- Benefits:
 - Smaller model size
 - Faster computation
 - Lower memory usage

2. Knowledge Distillation

- Knowledge distillation involves training a **smaller model using the knowledge of a larger model**.
- Large model → **Teacher model**
- Small model → **Student model**
- The student learns from the teacher's predictions and mimics its behavior.
- Benefits:
 - Smaller model size
 - Faster inference
 - Good performance

3. Weight Sharing

- In weight sharing, multiple parameters share the same value instead of storing separate values.
- This reduces storage requirements.
- Example:
Instead of storing 100 different weights, the model stores 10 shared weights.

4. Pruning

- Pruning is also considered a model compression technique because it reduces the number of parameters.

Deployment Efficiency

- Deployment efficiency means running AI models efficiently in real-world environments.
- Goals include:
- Faster prediction (low latency)
- Reduced memory usage
- Lower power consumption
- Ability to run on low-resource devices
- Pruning and compression help achieve these goals.

Example in Real Systems

- Large models such as:
- BERT
- GPT
- are often compressed before deployment so they can run on:
- smartphones
- smart assistants
- edge AI devices



Thank You